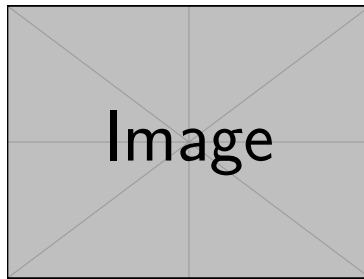


System Design Document

Self-Hosted Enterprise RAG Agent Platform

An Evolvable Architecture for Low-Latency, High-Throughput, and Trustworthy AI



Prepared by: The Engineering Team

Version: 3.0 (Final)

Date: August 2, 2026

Classification: Internal / Confidential

This document outlines the phased architectural evolution, deep-dive component designs, and critical system trade-offs for a fully containerized, on-premise RAG agent stack.

Contents

1	Introduction and System Overview	3
1.1	System Vision	3
1.2	Non-Functional Requirements	3
2	Phased Architectural Evolution	4
2.1	Phase 1: The MVP (Proof of Concept)	4
2.1.1	Architectural Snapshot	4
2.1.2	Components	4
2.1.3	Design Trade-offs	4
2.2	Phase 2: Production Readiness	4
2.2.1	Architectural Shift	4
2.2.2	Design Trade-offs	5
2.3	Phase 3: Scale, Quality & Autonomy	5
2.3.1	Architectural Maturity	5
2.3.2	Design Trade-offs	6
3	Deep-Dive: Internal Component Architectures	7
3.1	Subgraph A: The LangGraph Agent Core	7
3.1.1	Description	7
3.1.2	Node-by-Node Flow	7
3.1.3	Design Trade-off	7
3.2	Subgraph B: Fusion Retrieval System	8
3.2.1	Description	8
3.2.2	Process Steps	8
3.2.3	Design Trade-off	8
3.3	Subgraph C: Layered Memory System	8
3.3.1	Description	8
3.3.2	Tiers	8
3.3.3	Design Trade-off	9
3.4	Subgraph D: MCP Server (Model Context Protocol)	9
3.4.1	Description	9
3.4.2	Execution Pipeline	9
3.4.3	Design Trade-off	9
3.5	Subgraph E: Load Management & Autoscaling	9
3.5.1	Description	9
3.5.2	The Flow	10
3.5.3	Design Trade-off	10
3.6	Subgraph F: Self-Hosted LLM Router	10
3.6.1	Description	10
3.6.2	Routing Logic	10
3.6.3	Design Trade-off	10
3.7	Subgraph G: Observability & Evaluation Stack	11
3.7.1	Description	11

3.7.2	The Three Pillars	11
3.7.3	Evaluation (RAGAS)	11
3.7.4	Design Trade-off	11
4	Cross-Cutting Operational Concerns	12
4.1	Latency & Throughput Optimization	12
4.2	Guardrails Implementation (Input & Output)	12
4.3	Containerization & Orchestration (Kubernetes)	12
4.3.1	Resource Allocation QoS	12
4.3.2	Persistent Volumes	13
4.3.3	Service Mesh (Istio)	13
5	Conclusion and Summary of Design Decisions	14
A	Glossary of Terms	15

1. Introduction and System Overview

1.1 System Vision

This document describes the architecture of a **production-grade, self-hosted Retrieval-Augmented Generation (RAG) Agent platform**. The system is designed to process complex user queries by combining large language models (LLMs) with enterprise data, external tool calling, and sophisticated reasoning loops.

Key drivers for this design include:

- **Data Sovereignty:** 100% self-hosted infrastructure, utilizing open-weight models (e.g., Llama 3, Mistral) and open-source databases to ensure no external data leakage.
- **Performance:** Strict latency Service Level Objectives (SLOs) targeting a p95 latency of under 5 seconds for complex queries, with sustained throughput of 100 requests per second.
- **Quality:** Built-in guardrails, self-correction loops, and offline RAG evaluation to maintain high faithfulness (low hallucination rates).
- **Autonomy:** Event-driven architecture with auto-scaling to handle peak loads seamlessly.

1.2 Non-Functional Requirements

The architectural decisions are heavily driven by the following non-functional requirements:

- **Latency:** $p50 < 2s$, $p95 < 5s$, $p99 < 10s$ for end-to-end requests.
- **Throughput:** Sustained 100 req/sec, Burst 500 req/sec.
- **Availability:** 99.9% uptime SLA, achieved via circuit breakers and graceful degradation.
- **Scalability:** Horizontal scaling of stateless workers driven by queue depth.

2. Phased Architectural Evolution

To mitigate risk and allow for iterative learning, the platform is deployed in three distinct phases. Each phase introduces new capabilities while maintaining backward compatibility.

2.1 Phase 1: The MVP (Proof of Concept)

2.1.1 Architectural Snapshot

Phase 1 establishes the baseline. It is a **synchronous** FastAPI application that strictly handles requests sequentially.

2.1.2 Components

- **API Layer:** Synchronous FastAPI Gateway.
- **LLM:** Single instance of Llama 3 (70B) served locally via Ollama/vLLM.
- **Retrieval:** Simple vector similarity search using PostgreSQL with pgvector.
- **Memory:** Basic conversation history stored in PostgreSQL.
- **Caching:** Exact-match Redis cache.

2.1.3 Design Trade-offs

Phase 1 Trade-off: Simplicity vs. Scalability

Pro: Extremely simple to debug and deploy. Minimal moving parts guarantee a fast time-to-market.

Con: The synchronous nature blocks the server thread. A sudden spike in traffic causes timeouts (503 errors). The single LLM endpoint becomes a bottleneck, and there is zero observability (no tracing or metrics) to debug failures.

Decision: Accepted for initial validation, but explicitly designed to be replaced in Phase 2.

2.2 Phase 2: Production Readiness

2.2.1 Architectural Shift

The system is refactored to an **asynchronous, event-driven** architecture.

- **Decoupling:** A self-hosted Kafka cluster is introduced between the Gateway and Workers.
- **Orchestration:** The monolithic Agent is replaced by a **LangGraph** based multi-step executor (Plan, Retrieve, Reason, Verify).

- **Observability:** Introduction of OpenTelemetry, Prometheus, and Loki for the first time.
- **Hybrid Retrieval:** Neo4j is added alongside pgvector to capture graph relationships.

2.2.2 Design Trade-offs

Phase 2 Trade-off: Complexity vs. Resilience

Pro: The Kafka queue acts as a massive buffer, absorbing burst traffic. If workers crash, messages persist and are replayed. The introduction of the Verifier node (citation check) reduces hallucination rates significantly.

Con: Operational complexity skyrockets. Teams must now manage Kafka brokers, Jaeger instances, and a distributed tracing architecture. End-to-end latency increases by 200ms due to Kafka serialization.

Decision: Required for enterprise reliability. The latency hit is accepted in exchange for durability and crash recovery.

2.3 Phase 3: Scale, Quality & Autonomy

2.3.1 Architectural Maturity

The final phase focuses on intelligence and self-healing.

- **Advanced Retrieval:** Elasticsearch is added for BM25 (keyword) search, combined with a Cross-Encoder Reranker to fuse vector, graph, and keyword results.
- **Self-Correction:** The Agent loop gains a *Self-Correction* node that re-prompts the LLM on hallucination detection, creating a closed feedback loop.
- **Standardized Tooling:** Introduction of the Model Context Protocol (MCP) Server to standardize function calling (SQL, Web, Code, Internal APIs).
- **Autoscaling:** KEDA (Kubernetes Event-Driven Autoscaling) is deployed to scale workers based on Kafka consumer lag.
- **Quality Gates:** RAGAS evaluation pipelines run offline against historical data, triggering alerts on drift.

2.3.2 Design Trade-offs

Phase 3 Trade-off: Accuracy (Quality) vs. Computational Cost

Pro: The Self-Correction loop can reduce factual hallucination errors by an estimated 40-50%. The MCP server enables secure and auditable tool usage. The Cross-Encoder reranker drastically improves the precision of retrieved context (i.e., the "top 5" chunks are highly relevant).

Con: The Self-Correction loop adds 1 to 2 extra LLM inference calls (increasing latency by 2-4 seconds). The Cross-Encoder is computationally heavy compared to simple cosine similarity.

Decision: Implement *adaptive* triggering. The Self-Correction only fires if the Verifier's confidence score is between 40% and 70%. If below 40%, the system defaults to an "I don't know" response to save cost.

3. Deep-Dive: Internal Component Architectures

This chapter opens the "black boxes" of the Phase 3 architecture, as visualized in the detailed system diagrams.

3.1 Subgraph A: The LangGraph Agent Core

3.1.1 Description

This is the brains of the operation. It moves beyond a simple "single generation" call to implement an iterative, multi-step **ReAct** (Reason + Act) loop with Chain-of-Thought (CoT).

3.1.2 Node-by-Node Flow

1. **Planner:** Decomposes complex user queries into atomic sub-tasks. It enforces a hard limit on iterations (e.g., 3) to prevent infinite loops.
2. **Retriever Node:** Invokes the Fusion Retrieval system (detailed in B) to fetch relevant documents from the knowledge base.
3. **Reasoner (CoT):** Generates a thought trace. It explicitly states "I need X to answer Y" before requesting actions.
4. **Tool Node:** Calls the MCP Server (detailed in D) to execute functions like SQL queries or web searches.
5. **Verifier:** Cross-checks the proposed response against the retrieved context to calculate a confidence score.
6. **Self-Correction (Critical):** If the Verifier fails, this node sends the error log back to the Reasoner with a prompt correction, effectively acting as a "failure recovery" mechanism.

3.1.3 Design Trade-off

State Persistence vs. Latency

Maintaining the state graph across multiple iterations requires saving intermediate variables. If stored in PostgreSQL (disk), latency increases; if stored in Redis (memory), it's fast but volatile. **Decision:** Redis is used for working memory to keep the loop snappy, with PostgreSQL snapshots taken only at the end of the transaction.

3.2 Subgraph B: Fusion Retrieval System

3.2.1 Description

A high-recall, high-precision retrieval engine that avoids the "single search" trap.

3.2.2 Process Steps

1. **Query Processing:** HyDE (Hypothetical Document Embeddings) generates an imaginary ideal answer to broaden search scope. 2. **Parallel Fan-Out:** The query is sent simultaneously to: - *pgvector* (Semantic Similarity). - *Elasticsearch* (BM25 Keyword Matching). - *Neo4j* (Relationship Traversal). 3. **Reciprocal Rank Fusion (RRF):** Combines the ranking lists from the three sources into a single list (typically fetching 20 results). 4. **Cross-Encoder Reranking:** Heavy lifting. A dedicated local model (e.g., BGE-Reranker) scores each of the 20 (query, chunk) pairs pairwise. This reorders the list based on deep semantic relevance. 5. **Top-K Selection:** Only the top 5 chunks are passed to the Agent.

3.2.3 Design Trade-off

Parallelism vs. Resource Exhaustion

Fanning out to 3 databases simultaneously increases throughput (reduces waiting time) but opens 3x the database connections per request. **Decision:** Connection pooling is strictly limited on the database side, and query timeouts are set to 500ms to prevent long-tail delays slowing down the whole pipeline.

3.3 Subgraph C: Layered Memory System

3.3.1 Description

Humans have short-term and long-term memory. This system replicates that.

3.3.2 Tiers

- **Tier 1 - Working (Redis, TTL 15m):** Stores the last 5 conversation turns and the outputs of tool calls for the active session.
- **Tier 2 - Episodic (PostgreSQL):** Stores the complete immutable history of all conversations.
- **Tier 3 - Semantic (pgvector):** Extracts and stores factual embeddings about the user (e.g., "User prefers concise answers," "User is in the sales department").
- **Tier 4 - Procedural (MCP Registry):** Stores the available tools, their schemas, and access controls.

3.3.3 Design Trade-off

Context Window Usage

Including all 4 tiers in every prompt would exceed the LLM's 128k context window and drastically increase token cost. **Decision:** The *Context Aggregator* applies a relevancy threshold using vector similarity for Tiers 3 and 4 before injecting them into the prompt.

3.4 Subgraph D: MCP Server (Model Context Protocol)

3.4.1 Description

A standardized interface for tool calling, ensuring the LLM doesn't speak directly to raw APIs.

3.4.2 Execution Pipeline

1. **Validation:** Incoming tool requests (JSON) are validated against a strict JSON Schema to prevent injection attacks.
2. **Rate Limiting:** Enforced per tool per user to prevent abuse (e.g., limiting SQL executions to 10/min).
3. **Circuit Breaker:** Monitors the health of the underlying service (e.g., the SQL Executor). If it fails 3 times consecutively, the circuit opens and returns a graceful error without hitting the service.
4. **Sandboxed Execution:** The Code Interpreter runs inside a Docker container (using Pyodide) to ensure absolute isolation from the host kernel.

3.4.3 Design Trade-off

Standardization vs. Overhead

Implementing MPC adds a network hop and JSON serialization overhead (~50ms). **Decision:** Acceptable because it provides audit logging, security, and a unified interface. Without MCP, managing 20+ different API keys and authentication methods per service would be operationally unsustainable.

3.5 Subgraph E: Load Management & Autoscaling

3.5.1 Description

Ensures the system doesn't crumble under traffic spikes.

3.5.2 The Flow

1. **Ingress:** NGINX balances initial traffic.
2. **Rate Limiting:** Redis Sliding Window prevents a single user from DDOS-ing the system.
3. **Circuit Breaker:** If the system fails to respond to health checks, it returns a cached response instantly (Degraded Mode).
4. **Queuing:** Healthy requests are pushed to a Kafka topic.
5. **Autoscaling (KEDA):** KEDA monitors the "lag" (unconsumed messages) in Kafka. If lag exceeds 100 messages per partition, it scales up worker pods. When lag drops, it scales down.

3.5.3 Design Trade-off

Cold Starts vs. Cost

Rapid scaling of GPU pods (for LLM inference) is expensive and slow. **Decision:** Implement "Target Buffer" scaling to keep a minimum of 2 hot pods running at all times, sacrificing a small amount of idle cost for significantly lower p99 latency during scale-up events.

3.6 Subgraph F: Self-Hosted LLM Router

3.6.1 Description

A smart routing layer that avoids sending "simple" queries to the massive, costly model.

3.6.2 Routing Logic

The router analyzes the request metadata (token length, detected intent) and routes to: - **Primary:** vLLM instance hosting Llama 3 70B (for complex reasoning, code generation). - **Fallback:** Ollama hosting Mistral 7B (for simple chat, summarization). - **Embedding:** vLLM hosting BGE (for vectorizing queries during retrieval).

3.6.3 Design Trade-off

Model Accuracy vs. Inference Speed

Llama 3 70B is highly accurate but has high latency (2-3s). Mistral 7B is faster (500ms) but less capable. **Decision:** Route is based on user tier (Premium gets Llama, Free gets Mistral) and complexity threshold measured by token count. This optimizes average throughput while preserving high quality for complex tasks.

3.7 Subgraph G: Observability & Evaluation Stack

3.7.1 Description

A self-hosted "nerve center" enabling teams to monitor and improve the system continuously.

3.7.2 The Three Pillars

1. **Traces (Jaeger):** Captures every span (e.g., "LLM Call," "Retrieval," "Verifier"). Essential for debugging "Why was this request slow?"
2. **Metrics (Prometheus):** Gauges and Histograms tracking p95 latency, cache hit ratio, and hallucination rate. Alert rules fire if error rates exceed 2%.
3. **Logs (Loki):** Structured JSON logs with Correlation IDs for auditing every user interaction and guardrail violation.

3.7.3 Evaluation (RAGAS)

An offline cronjob runs RAGAS benchmarks (Faithfulness, Answer Relevancy, Context Precision) on historical query data. The "Drift Detection" module compares today's scores against last week's. A drop $> 10\%$ triggers an alert to retrain or re-index the vector database.

3.7.4 Design Trade-off

Data Volume vs. Storage Cost

Instrumenting every request generates massive amounts of trace data (TB per day). **Decision:** Probabilistic sampling (10% of traces) is applied in Phase 3. However, traces containing errors are explicitly sampled at 100% to ensure zero loss of failure data.

4. Cross-Cutting Operational Concerns

4.1 Latency & Throughput Optimization

Achieving the target throughput of 100 req/sec requires a multi-layered strategy:

1. **Database Connection Pools:** Each pod maintains a persistent pool of connections to PostgreSQL and Neo4j to avoid TCP handshake overhead.
2. **Asynchronous I/O:** The Python workers utilize `async/await` to handle database calls concurrently while the LLM is generating tokens.
3. **Caching:** The two-tier cache (Exact + Semantic) targets a 60% hit ratio, which bypasses the entire heavy retrieval and Agent loop for 60% of requests, dramatically reducing average latency.
4. **Token Streaming:** For the user-facing frontend, tokens are streamed via Server-Sent Events (SSE) while the Agent runs, improving Time-To-First-Token (TTFT) perceived by the user.

4.2 Guardrails Implementation (Input & Output)

A robust policy engine is essential for safe deployment.

- **Input Guardrails (NeMo Guardrails):** Located just after the cache miss check. This engine scans the user prompt for: - Prompt Injection patterns (e.g., "Ignore your system prompt..."). - PII (Personal Identifiable Information - SSN, emails) using Regex. - Toxic or unsupported topics.
- **Output Guardrails:** Located before the response is sent to the client. It scans the response for: - Hallucinated citations (checks if the cited source ID actually exists in the retrieved context). - Leaked secrets (e.g., API keys in code output). - Phishing or malicious URLs.
- **Fallback Action:** If a guardrail is tripped, the system returns a sanitized response (e.g., "I am unable to answer this question") or logs an "Audit" violation.

4.3 Containerization & Orchestration (Kubernetes)

All components run as containers on a dedicated Kubernetes cluster, configured via Helm charts.

4.3.1 Resource Allocation $\parallel QoS$

- **Guaranteed Class:** GPU Workers (vLLM) and Kafka brokers are assigned strict `requests` and `limits` to ensure they never compete for resources, preventing OOM (Out of Memory) kills.

- **Burstable Class:** Stateless FastAPI workers and the Neo4j instances use burstable QoS, allowing them to consume free resources during low traffic but throttling them during high traffic.

4.3.2 Persistent Volumes

PostgreSQL and Elasticsearch use StatefulSets with PersistentVolumeClaims (PVCs) backed by local SSDs to ensure high IOPS for vector searches.

4.3.3 Service Mesh (Istio)

Istio is deployed to manage mTLS encryption between services (e.g., MCP Server ↔ Internal APIs) and to provide fine-grained traffic routing (e.g., sending 10% of traffic to a new model version for A/B testing).

5. Conclusion and Summary of Design Decisions

The evolution from a synchronous MVP to a self-correcting, event-driven enterprise system requires navigating significant trade-offs between complexity, latency, and accuracy.

This document has delineated a clear pathway to manage these trade-offs:

1. **Trade Complexity for Resilience:** Asynchronous queuing and circuit breakers are non-negotiable for a production service, despite the operational overhead.
2. **Trade Speed for Accuracy:** Cross-Encoder reranking and the Self-Correction loop add latency to critical paths but are essential to guarantee the "trustworthiness" of the AI. We mitigate this by making correction *adaptive*.
3. **Embrace Observability as a Core Component:** Monitoring and Evaluation are not "add-ons" but intrinsic design constraints that dictate how the system behaves and evolves.

The final architecture presented is a robust, self-contained "brain" for enterprise AI—fully containerized, horizontally scalable, and secure. It stands ready to handle enterprise-scale workloads while maintaining strict data sovereignty.

A. Glossary of Terms

Term	Description
CoT	Chain-of-Thought: A prompting strategy that encourages the LLM to show its reasoning step-by-step.
HyDE	Hypothetical Document Embeddings: Generates a fake answer to query to improve retrieval retrieval quality.
KEDA	Kubernetes Event-Driven Autoscaling: Scales pods based on external events (e.g., queue length).
MCP	Model Context Protocol: A standardized protocol for exposing tools to LLMs.
RRF	Reciprocal Rank Fusion: A method to combine multiple ranked result lists.
RAGAS	Retrieval-Augmented Generation Assessment: A framework for evaluating RAG pipelines.
SLO	Service Level Objective: A target metric the system must meet (e.g., p95 latency).
vLLM	A high-throughput and memory-efficient inference engine for LLMs.